

NIFTY AI Trader — 10 Phase Build Plan for Google Jules

Optimized for Google Jules (Async Coding Agent) • Gemini 2.5 Flash in-app

HOW TO USE WITH JULES

1. Create an empty GitHub repo named `nifty-ai-trader`
2. Connect it to Jules at `jules.google`
3. Paste each phase prompt as a new Jules task
4. Jules will present a plan — review it, then approve
5. After Jules opens a PR, merge it before starting the next phase
6. Do NOT skip phases — each one builds on the previous

PHASE 1 — Project Bootstrap, Architecture & Theme System

TASK: Bootstrap a complete Flutter project for a NIFTY options trading assistant app.

This is Phase 1 of a 10-phase build. The repo is currently empty. By the end of this task, the project must compile and run with zero errors showing a shell navigation UI.

STEP 1: FLUTTER PROJECT STRUCTURE

Create the project with ``flutter create nifty_ai_trader`` then organize lib/ like this:

```
lib/
├─ core/
│   ├─ config/
│   │   └─ app_config.dart           ← reads env vars via --dart-define
│   ├─ constants/
│   │   └─ app_constants.dart       ← intervals, timeouts, market hours
│   └─ theme/
│       └─ app_theme.dart           ← dark + light MaterialTheme
└─ utils/
    ├─ date_utils.dart              ← IST helpers, market hours checker
    └─ price_formatter.dart         ← ₹ formatting with commas
```

```
└─ data/
  │ └─ models/                ← empty for now, populated in Phase 2
  │ └─ repositories/         ← empty for now
  │   └─ sources/            ← empty for now
└─ features/
  │ └─ dashboard/
  │   │ └─ dashboard_screen.dart ← placeholder
  │   └─ signals/
  │     │ └─ signals_screen.dart ← placeholder
  │     └─ trade_tracker/
  │       │ └─ trade_tracker_screen.dart ← placeholder
  │       └─ pnl/
  │         │ └─ pnl_screen.dart      ← placeholder
  │         └─ settings/
  │           └─ settings_screen.dart ← placeholder
└─ main.dart
```

STEP 2: PUBSPEC.YAML – ADD ALL DEPENDENCIES

```
dependencies:
  flutter:
    sdk: flutter
  # State management
  flutter_riverpod: ^2.5.1
  riverpod_annotation: ^2.3.5
  # Navigation
  go_router: ^13.2.0
  # Networking
  http: ^1.2.2
  web_socket_channel: ^2.4.5
  # AI
  google_generative_ai: ^0.4.6
  # Charts
  fl_chart: ^0.68.0
  # Notifications
  firebase_core: ^3.3.0
  firebase_messaging: ^15.0.4
  flutter_local_notifications: ^17.2.1
  # Storage
  hive_flutter: ^1.1.0
  shared_preferences: ^2.3.2
  # Audio
  audioplayers: ^6.0.0
  # Utilities
  intl: ^0.19.0
```

```
connectivity_plus: ^6.0.3
path_provider: ^2.1.4

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^4.0.0
  riverpod_generator: ^2.4.3
  build_runner: ^2.4.12

flutter:
  uses-material-design: true
  fonts:
    - family: JetBrainsMono
      fonts:
        - asset: assets/fonts/JetBrainsMono-Regular.ttf
        - asset: assets/fonts/JetBrainsMono-Bold.ttf
          weight: 700
  assets:
    - assets/sounds/
    - assets/fonts/
```

STEP 3: APP CONFIG – API KEYS VIA DART-DEFINE

In lib/core/config/app_config.dart:

```
class AppConfig {
  static const String dhanClientId =
    String.fromEnvironment('DHAN_CLIENT_ID', defaultValue: '');
  static const String dhanAccessToken =
    String.fromEnvironment('DHAN_ACCESS_TOKEN', defaultValue: '');
  static const String geminiApiKey =
    String.fromEnvironment('GEMINI_API_KEY', defaultValue: '');

  // App run with:
  // flutter run --dart-define=DHAN_CLIENT_ID=xxx --dart-
define=DHAN_ACCESS_TOKEN=yyy --dart-define=GEMINI_API_KEY=zzz
  // NEVER hardcode keys in source code
}
```

STEP 4: THEME SYSTEM – DARK FIRST, MATERIAL 3

In lib/core/theme/app_theme.dart build TWO ThemeData objects:

Color palette (dark theme):

- scaffold background: Color(0xFF0A0E1A) ← deep navy
- surface: Color(0xFF131929)
- card: Color(0xFF1C2333)
- primary (green): Color(0xFF00C896) ← signal green
- error (red): Color(0xFFFFF4757)
- warning (amber): Color(0xFFFFFB800)
- text primary: Color(0xFFFF1F5F9)
- text secondary: Color(0xFF94A3B8)

Custom text theme:

- priceStyle: JetBrainsMono, size 28, weight bold (for live prices)
- monoStyle: JetBrainsMono, size 14 (for OHLCV values)

Light theme: standard Material 3 light with primary Color(0xFF00C896)

STEP 5: ROUTING & MAIN APP

Set up GoRouter with shell route (bottom nav) containing 5 tabs:

- /dashboard → DashboardScreen
- /signals → SignalsScreen
- /trade → TradeTrackerScreen
- /pnl → PnLScreen
- /settings → SettingsScreen

Bottom navigation icons: dashboard, notifications, track_changes, bar_chart, settings

Wrap MaterialApp.router with ProviderScope.

EXPECTED OUTPUT:

- `flutter run` succeeds with zero errors
- App shows 5-tab bottom navigation
- Each tab shows a centered placeholder Text with the screen name
- Dark theme applied by default
- No hardcoded API keys anywhere in source

PHASE 2 — Dhan API Client & WebSocket Live Price Feed

TASK: Implement the complete Dhan API integration layer – REST client for historical

candle data and WebSocket client for real-time NIFTY price ticks.

Phase 1 is complete. The project runs with placeholder screens and all packages installed. This phase builds the entire data layer.

STEP 1: DATA MODELS

Create lib/data/models/candle.dart:

```
class Candle {
  final DateTime timestamp;
  final double open, high, low, close, volume;
  bool get isBullish => close >= open;
  double get bodySize => (close - open).abs();
}
Include: fromJson(Map), toJson(), copyWith()
```

Create lib/data/models/nifty_tick.dart:

```
class NiftyTick {
  final double price, change, changePercent;
  final DateTime timestamp;
  final bool isPositive;
}
```

STEP 2: DHAN REST API CLIENT

Create lib/data/sources/dhan_api_client.dart:

Base URL: <https://api.dhan.co>

Auth header: Authorization: Bearer {AppConfig.dhanAccessToken}

Implement these methods:

```
1. Future<List<Candle>> getHistoricalCandles({
  required String securityId,    // "13" for NIFTY 50 index
  required String exchangeSegment, // "IDX_I"
  required String instrument,    // "INDEX"
  required DateTime fromDate,
  required DateTime toDate,
  String interval = "5",        // "1", "5", "15", "60", "D"
})
→ POST /v2/charts/intraday
→ Parse "open", "high", "low", "close", "volume", "timestamp" arrays from
response
```

→ Return List<Candle> sorted ascending by timestamp

2. Future<double> getCurrentPrice(String securityId)

→ GET /v2/marketfeed/quote

→ Return last traded price for NIFTY 50

3. Handle HTTP errors:

- 401: throw AuthException("Dhan token expired or invalid")
- 429: throw RateLimitException("Too many requests")
- 5xx: throw ServerException("Dhan server error: \$statusCode")
- network error: throw NetworkException(message)

STEP 3: WEBSOCKET SERVICE – REAL-TIME TICKS

Create lib/data/sources/dhan_websocket_service.dart:

```
class DhanWebSocketService {
  // Dhan Market Feed WebSocket endpoint
  // Connects and subscribes to NIFTY 50 (securityId: "13", exchangeSegment:
  IDX_I)

  Stream<NiftyTick> get priceStream → broadcasts parsed ticks

  ConnectionStatus get status → CONNECTING / CONNECTED / DISCONNECTED /
  RECONNECTING

  Future<void> connect() ← call on app start
  Future<void> disconnect() ← call on dispose

  // RECONNECTION LOGIC (mandatory):
  // On any disconnect: wait 2s → reconnect
  // If reconnect fails: exponential backoff (2, 4, 8, 16, 32s max)
  // After 5 consecutive failures: switch to REST polling fallback (every 15s)
  // Expose connectionStatusStream: Stream<ConnectionStatus>
}
```

STEP 4: RIVERPOD PROVIDERS

Create lib/features/dashboard/providers/market_providers.dart:

1. websocketServiceProvider: Provider<DhanWebSocketService>
(initialized and connected on first access)

2. `livePriceProvider: StreamProvider<NiftyTick>`
 - wraps `websocketServiceProvider.priceStream`
 - falls back to polling via `dhanApiClient` if `WebSocket` unavailable
3. `historicalCandlesProvider: FutureProvider<List<Candle>>`
 - fetches last 60 candles of 5-min data
 - auto-refreshes every 5 minutes using `ref.invalidateSelf() + Timer`
4. `connectionStatusProvider: StreamProvider<ConnectionStatus>`
 - wraps `websocketServiceProvider.connectionStatusStream`
5. `isMarketOpenProvider: Provider<bool>`
 - uses `DateUtils.isMarketOpen()` – true Mon-Fri 9:15-15:30 IST

STEP 5: TEST WIDGET IN DASHBOARD SCREEN

Update `DashboardScreen` (temporary, will be replaced in Phase 6) to show:

- "● LIVE" / "○ RECONNECTING" / "■ OFFLINE" status indicator
- Current NIFTY price from `livePriceProvider` in `JetBrainsMono` font
- Last 5 candles as text (timestamp | O | H | L | C)
- "Market OPEN" / "CLOSED" label

EXPECTED OUTPUT:

- Dashboard tab shows live NIFTY price updating in real time
- Price flashes green/red on change
- Connection indicator shows LIVE when `WebSocket` connected
- If `WebSocket` fails, falls back to 15s REST polling automatically
- Zero unhandled exceptions during 10 minutes of running

PHASE 3 — Technical Indicator Engine (Pure Dart)

TASK: Build a complete technical indicator calculation engine in pure Dart. No external packages – all math implemented from scratch for full accuracy.

Phase 2 is complete. `historicalCandlesProvider` returns `List<Candle>`.

STEP 1: INDICATOR SERVICE

Create `lib/data/sources/indicator_service.dart` with these exact methods:

```

// EMA using standard formula:  $k = 2 / (\text{period} + 1)$ ,  $\text{EMA} = \text{price} * k + \text{prevEMA} * (1 - k)$ 
// First EMA value = SMA of first `period` values
List<double?> calculateEMA(List<double> prices, int period)

// MACD = EMA(12) - EMA(26), Signal = EMA(9) of MACD, Histogram = MACD - Signal
MACDData calculateMACD(List<double> prices, {int fast=12, int slow=26, int
signal=9})

// RSI using Wilder's smoothing method (standard for TradingView compatibility)
// Returns only the LATEST RSI value as double
double calculateRSI(List<double> prices, {int period=14})

// Volume analysis
VolumeData analyzeVolume(List<Candle> candles)

```

STEP 2: OUTPUT MODELS

Create lib/data/models/indicator_result.dart:

```

class MACDData {
  final List<double?> macdLine;      // length = input length
  final List<double?> signalLine;
  final List<double?> histogram;
  final double latestMACD;
  final double latestSignal;
  final double latestHistogram;
  final MACDSignal signal;          // enum below
}

enum MACDSignal {
  bullishCross,    // histogram crossed from negative to positive this candle
  bearishCross,    // histogram crossed from positive to negative this candle
  bullish,         // histogram positive, no cross
  bearish,         // histogram negative, no cross
}

class VolumeData {
  final double currentVolume;
  final double avgVolume20;        // 20-candle average
  final double volumeRatio;        // currentVolume / avgVolume20
  final bool isVolumeSpike;        // volumeRatio > 1.5
}

class IndicatorResult {
  final double ema9, ema21, ema50;
}

```



```

final MACDData macd;
final double rsi;
final VolumeData volume;
final EMASStack emaStack;          // BULLISH / BEARISH / MIXED
final DateTime calculatedAt;
}

enum EMASStack {
  bullish,    // ema9 > ema21 > ema50
  bearish,    // ema9 < ema21 < ema50
  mixed,
}

```

STEP 3: RIVERPOD PROVIDER

Create indicatorResultProvider: Provider<AsyncValue<IndicatorResult>>

- Watch historicalCandlesProvider
- When candles update: run ALL calculations inside compute() isolate (use Flutter's compute() function – indicators are CPU-heavy)
- Return null / loading state if fewer than 52 candles (not enough data)
- Log calculation time in debug mode

STEP 4: UNIT TESTS (MANDATORY)

Create test/indicator_service_test.dart:

Test 1: EMA calculation

- Input: [10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20], period=5
- Expected last EMA: approximately 17.03 (verify against TradingView)

Test 2: MACD crossover detection

- Create a synthetic price series that clearly crosses
- Verify MACDSignal.bullishCross is detected at the correct candle

Test 3: RSI boundary conditions

- 14 identical prices → RSI = 50.0
- Only up days for 14 periods → RSI = 100.0
- Only down days → RSI = 0.0

Test 4: EMASStack detection

- ema9 > ema21 > ema50 → EMASStack.bullish
- ema9 < ema21 < ema50 → EMASStack.bearish

EXPECTED OUTPUT:

- All 4 tests pass with flutter test
- Debug screen shows live indicator values updating every 5 minutes
- EMA and RSI values within $\pm 0.5\%$ of TradingView for same candles

PHASE 4 — Gemini AI Signal Engine

TASK: Build the AI-powered signal generation engine using `google_generative_ai` package with the `gemini-2.5-flash` model. This is the core intelligence of the app.

Phase 3 is complete. `IndicatorResult` is computed and available via Riverpod.

STEP 1: TRADE SIGNAL MODEL

Create `lib/data/models/trade_signal.dart`:

```
enum SignalAction {
  buyPE,      // BUY_PE
  buyCE,      // BUY_CE
  exit50,     // EXIT_50
  exit100,    // EXIT_100
  trailSL,    // TRAIL_SL
  hold,       // HOLD
  noTrade,    // NO_TRADE
}

enum SignalConfidence { high, medium, low }

class TradeSignal {
  final String id;                // UUID
  final SignalAction action;
  final int? strike;              // e.g. 23800
  final String? optionType;       // "PE" or "CE"
  final double? price;            // entry/exit price
  final double? sl;               // stop loss
  final double? tp1;              // target 1
  final double? tp2;              // target 2
  final String reason;            // AI explanation (max 60 words)
  final SignalConfidence confidence;
  final DateTime timestamp;
  final double niftyPriceAtSignal; // NIFTY level when signal was generated
```

```
bool get isActionable =>
  action != SignalAction.hold && action != SignalAction.noTrade;
}
```

STEP 2: GEMINI SERVICE

Create lib/data/sources/gemini_service.dart:

```
Use package: google_generative_ai
Model: "gemini-2.5-flash"
API key: AppConfig.geminiApiKey
```

Safety settings: set all harm categories to BLOCK_NONE
(financial analysis prompts can trip safety filters)

Method: Future<TradeSignal> analyzeMarket(AnalysisInput input)

Build this EXACT system prompt:

```
"""
You are an expert NIFTY 50 options trading analyst. You respond ONLY with a single
valid JSON object. No markdown, no explanation, no code blocks. Just the raw JSON.
```

JSON schema (follow exactly):

```
{
  "action": "BUY_PE|BUY_CE|EXIT_50|EXIT_100|TRAIL_SL|HOLD|NO_TRADE",
  "strike": 23800,
  "optionType": "PE|CE|null",
  "price": 185.50,
  "sl": 163.00,
  "tp1": 215.00,
  "tp2": 245.00,
  "reason": "< 60 words explaining the signal >",
  "confidence": "HIGH|MEDIUM|LOW"
}
```

Rules:

- Only trade between 9:30 AM and 2:30 PM IST. Outside these hours: NO_TRADE.
 - SL for PE buy must be at least 12% below entry price
 - SL for CE buy must be at least 12% below entry price
 - Only generate BUY signal when confidence is HIGH or MEDIUM
 - TRAIL_SL: set price = new stop loss level (not entry)
 - HOLD / NO_TRADE: set all price fields to null
- ```
"""
```

User message template (fill from AnalysisInput):

```
"""
```

```
Market Snapshot – {time} IST
```

```
NIFTY 50: ₹{niftyPrice} | Day change: {changePercent}%
```

```
Technical Indicators:
```

- EMA Stack: {emaStack} → EMA9:{ema9} | EMA21:{ema21} | EMA50:{ema50}
- MACD: {macdLine} | Signal: {signalLine} | Hist: {histogram} | Cross: {macdSignal}
- RSI(14): {rsi}
- Volume: {volumeRatio}x 20-period average {volumeSpike}

```
Last 5 candles (5-min, newest last):
```

```
{candleTable}
```

```
{activeTrade}
```

```
"""
```

Where {activeTrade} is either empty or:

```
"Active Position: {strike} {type} @ ₹{entry} | CMP: ₹{cmp} | P&L: {pnlPct}% | SL: ₹{sl}"
```

---

### STEP 3: SIGNAL PARSER

---

Create lib/data/sources/signal\_parser.dart:

```
class SignalParser {
 TradeSignal parse(String geminiResponse, double niftyPrice) {
 // 1. Strip any accidental markdown fences (```json ... ```)
 // 2. jsonDecode
 // 3. Validate all required fields present and correct types
 // 4. Validate business rules:
 // - If action=BUY_PE: strike and sl must be non-null
 // - If action=TRAIL_SL: price must be non-null
 // - sl must be numerically lower than price for BUY signals
 // 5. Return TradeSignal
 // 6. On any error: throw ParseException with raw response for logging
 }
}
```

---

### STEP 4: SIGNAL SCHEDULER

---

Create lib/features/signals/signal\_scheduler.dart:

```

class SignalScheduler {
 // Emits a new TradeSignal every time a 5-minute candle closes
 // Aligned to real candle close times: 9:35, 9:40, 9:45... 15:25, 15:30
 // Logic: on start, calculate time to next 5-min boundary, wait, then repeat

 Stream<TradeSignal> get signalStream

 // Rate limiting: track Gemini API calls
 // Gemini Flash free: 15 RPM, 1500 RPD
 // If at 14 calls in last 60s: log and skip this cycle
 // Expose: lastCallAt, totalCallsToday, estimatedCostToday (₹)

 void start()
 void stop()
 void forceAnalysis() // manual trigger for testing
}

```

---

## STEP 5: SIGNAL HISTORY PROVIDER

---

signalHistoryProvider: StateNotifierProvider<List<TradeSignal>>

- Add new signal at front of list
- Keep max 100 in memory
- Persist each signal to Hive box "signals" as JSON string
- Load from Hive on app start

### EXPECTED OUTPUT:

- Every 5 min during market hours: Gemini is called, a valid TradeSignal is returned
- All 7 action types parse correctly
- HOLD and NO\_TRADE signals are not sent as notifications (handled in Phase 5)
- Signal history survives app restart (loaded from Hive)
- Zero crashes when Gemini returns malformed JSON (caught by SignalParser)

---

## PHASE 5 — Push Notifications & Smart Alert System

TASK: Implement the full notification and alert system. Every actionable signal must

reach the user via push notification + sound, even when the app is minimized.

Phase 4 is complete. TradeSignal flows from SignalScheduler into signalHistoryProvider.

---

## STEP 1: FIREBASE SETUP

---

- Create android/app/google-services.json placeholder with instructions
- Create ios/Runner/GoogleService-Info.plist placeholder
- Add Firebase initialization to main.dart:  
    await Firebase.initializeApp();
- Request notification permissions at app start (both Android 13+ and iOS)
- Save FCM token to SharedPreferences key "fcm\_token"

---

## STEP 2: NOTIFICATION SERVICE

---

Create lib/core/services/notification\_service.dart:

Notification channels (Android):

1. "trading\_signals" – HIGH importance (for new signals)
2. "sl\_alerts" – MAX importance + vibration (for SL breach)
3. "general" – DEFAULT importance (info messages)

Method: void showSignalNotification(TradeSignal signal)

Build title dynamically:

- BUY\_PE → "🔴 BUY {strike} PE @ ₹{price}"
- BUY\_CE → "🟢 BUY {strike} CE @ ₹{price}"
- EXIT\_50 → "📉 Exit 50% at ₹{price}"
- EXIT\_100 → "🚫 EXIT 100% at ₹{price}"
- TRAIL\_SL → "⚡ Trail SL to ₹{price}"

Body: "{signal.reason} | {signal.confidence} confidence"

Tap action: open /signals screen

Method: void showPriceAlert(String title, String body, {bool urgent = false})

Use "sl\_alerts" channel when urgent=true

---

## STEP 3: SOUND ALERTS

---

Create lib/core/services/sound\_service.dart using audioplayers:

Add 3 placeholder audio files to assets/sounds/:

- signal\_buy.mp3 (short sharp beep)
- signal\_exit.mp3 (double beep)
- sl\_breach.mp3 (urgent repeating alarm)

If real MP3 files aren't available, generate simple beep WAV files

programmatically

using Dart's `dart:typed_data` (sine wave generation) as placeholders.

Method: `Future<void> playSignalSound(TradeSignal signal)`

- BUY\_PE / BUY\_CE → `signal_buy.mp3`
- EXIT\_50 / EXIT\_100 / TRAIL\_SL → `signal_exit.mp3`
- HOLD / NO\_TRADE → no sound

Method: `Future<void> playUrgentAlarm()` → `sl_breach.mp3`

---

## STEP 4: ALERT RULE ENGINE

---

Create `lib/features/trade_tracker/alert_rule_engine.dart`:

Watches: `activeTrade` + `livePriceProvider`

On every price tick, check these rules in order:

Rule 1: SL Breach

```
if (activeTrade != null && currentPrice <= activeTrade.sl)
 → showPriceAlert("⚠ STOP LOSS BREACHED", "Exit NOW – SL at ₹{sl} hit", urgent:
true)
 → playUrgentAlarm()
 → mark alert as fired (don't repeat for same trade)
```

Rule 2: TP1 Hit

```
if (currentPrice >= activeTrade.tp1 && !tp1AlertFired)
 → showPriceAlert("✅ TP1 Hit!", "Price ₹{tp1} reached. Consider partial exit.")
 → tp1AlertFired = true
```

Rule 3: TP2 Hit

```
if (currentPrice >= activeTrade.tp2 && !tp2AlertFired)
 → showPriceAlert("🎯 TP2 Hit! Book full profits.", urgent: false)
 → tp2AlertFired = true
```

Rule 4: Down 8% Alert

```
if (unrealizedPnlPercent <= -8.0 && !down8AlertFired)
 → showPriceAlert("📉 Trade -8%", "Review position. Near SL territory.")
 → down8AlertFired = true
```

Reset all fired flags when trade is closed.

---

## STEP 5: WIRE EVERYTHING TOGETHER

---

In signalHistoryProvider's StateNotifier:

On add new signal (if isActionable):

- notificationService.showSignalNotification(signal)
- soundService.playSignalSound(signal)

Firebase background message handler (top-level function, not inside class):

```
@pragma('vm:entry-point')
```

```
Future<void> firebaseMessagingBackgroundHandler(RemoteMessage message) async {
 await Firebase.initializeApp();
 // show local notification from background
}
```

EXPECTED OUTPUT:

- New BUY signal → phone shows push notification with sound immediately
- SL breach → urgent alarm plays + notification appears even on lock screen
- FCM token is saved to SharedPreferences
- Background handler registered and logs received messages

---

## PHASE 6 — Dashboard UI: Live Chart & Signal Feed

TASK: Build the complete, production-quality Dashboard screen. This is the main screen

users see every time they open the app. It must look professional and feel fast.

Phases 1-5 complete. All data providers and notification services are working.

---

DESIGN TOKENS (use these exact values throughout)

---

|              |         |                                |
|--------------|---------|--------------------------------|
| Background:  | #0A0E1A | (deep navy)                    |
| Surface:     | #131929 | (card background)              |
| Card:        | #1C2333 | (elevated card)                |
| Primary:     | #00C896 | (green – bullish, buy signals) |
| Danger:      | #FF4757 | (red – bearish, exit signals)  |
| Warning:     | #FFB800 | (amber – trail SL, caution)    |
| TextPrimary: | #F1F5F9 |                                |
| TextMuted:   | #94A3B8 |                                |
| Border:      | #2D3748 | with 60% opacity               |

Price font: JetBrainsMono (configured in Phase 1)

---

WIDGET 1: MarketStatusBar (persistent, top of every screen)



---

Full-width bar, height 36px, surface color background.  
Left side: animated pulsing dot + "MARKET OPEN" (green) or "CLOSED" (gray)  
Center: current IST time updating every second  
Right side: "● LIVE" (green) / "⌂ RECONNECTING" (amber) / "× OFFLINE" (red)  
based on `connectionStatusProvider`

---

#### WIDGET 2: LivePriceHeader

---

"NIFTY 50" label (12px, muted)  
Price: "₹23,822.45" in JetBrainsMono 32px bold  
Change: "▼ -0.02% (-4.85)" in red or "▲ +0.45% (+107.20)" in green  
On price update: background flashes green/red for 400ms then fades back

---

#### WIDGET 3: CandlestickChart (using `fl_chart`)

---

Height: 220px

Main chart (70% height):

- Show last 30 candles
- Green candles:  $\text{close} \geq \text{open}$  (body color #00C896 at 80% opacity)
- Red candles:  $\text{close} < \text{open}$  (body color #FF4757 at 80% opacity)
- Wick lines: 1px, same color as body but 50% opacity
- Overlay EMA9 as blue line (`LineChartBarData`), EMA21 as orange line
- X-axis: time labels every 5 candles (HH:mm format)
- Y-axis: price labels, auto-range with 5% padding
- Tap on candle: show tooltip with O/H/L/C/Vol

MACD histogram (30% height, below main):

- Green bars when histogram  $> 0$ , red bars when  $< 0$
  - MACD line and signal line as overlaid thin lines
  - No x-axis (shares with main chart)
- 

#### WIDGET 4: LatestSignalCard

---

Full-width card with left border 4px colored by action:

- BUY\_PE / BUY\_CE → green border + green action badge
- EXIT\_\* → red border + red badge
- TRAIL\_SL → amber border + amber badge
- HOLD / NO\_TRADE → gray border

Layout:

Row 1: [Action badge "BUY 23800 PE"] [timestamp "2 min ago"] [confidence chip "HIGH"]

Row 2: "₹185.50" entry price large | "SL: ₹163" | "TP1: ₹215 / TP2: ₹245"

Row 3: Reason text in italic muted color (max 2 lines, ellipsis)

Pulse animation: if signal is < 5 minutes old, border animates with opacity 0.4→1.0→0.4

---

#### WIDGET 5: SignalHistoryList

---

Scrollable list of past signals (compact rows):

[Time HH:mm] [colored action chip] [reason truncated] [NIFTY price]

Header row: "Signal History" on left, signal count badge on right

---

#### DASHBOARD SCREEN LAYOUT (lib/features/dashboard/dashboard\_screen.dart)

---

CustomScrollView with:

SliverAppBar (MarketStatusBar pinned, no title)

SliverToBoxAdapter:

LivePriceHeader (padding 16px)

SizedBox(h: 12)

CandlestickChart (padding h: 16)

SizedBox(h: 12)

LatestSignalCard (padding h: 16) – show skeleton if no signals yet

SizedBox(h: 8)

"Signal History" header

SliverList:

SignalHistoryList items

Floating Action Button: "Analyze Now" → calls `signalScheduler.forceAnalysis()`

Show loading spinner while analysis is in progress

Disable during analysis to prevent double-taps

EXPECTED OUTPUT:

- Dashboard is visually polished and production-quality
- Live price updates animate smoothly without jank
- Candlestick chart renders all 30 candles with EMA overlays
- MACD histogram shows correctly in the sub-chart

- LatestSignalCard pulses for new signals
- FAB triggers a manual Gemini analysis

## PHASE 7 — Trade Tracker & P&L Dashboard

TASK: Build the complete trade management system – log trades, track live P&L, monitor SL/TP, view historical performance, all persisted in Hive.

Phases 1-6 complete. Alert system and live price feed working.

---

### STEP 1: DATA MODELS

---

Create lib/data/models/active\_trade.dart:

```
@HiveType(typeId: 1)
class ActiveTrade extends HiveObject {
 final String id; // UUID
 final String symbol; // "NIFTY"
 final int strike; // e.g. 23800
 final String optionType; // "PE" or "CE"
 final double entryPrice;
 double currentPrice; // mutable, updated by price feed
 final int lots;
 final int quantityPerLot; // 25 for NIFTY options
 double sl; // mutable (for trail SL)
 final double tp1;
 final double tp2;
 final DateTime entryTime;
 TradeStatus status; // OPEN / CLOSED / SL_HIT / TP1_HIT / TP2_HIT
 double? exitPrice;
 DateTime? exitTime;
 String? exitReason; // MANUAL / SL_HIT / TP1_HIT / TP2_HIT / AI_SIGNAL
 String? triggeringSignalId; // ID of TradeSignal that opened this trade

 // Computed (not stored):
 double get totalQuantity => lots * quantityPerLot;
 double get unrealizedPnl => (currentPrice - entryPrice) * totalQuantity;
 double get unrealizedPnlPercent => ((currentPrice - entryPrice) / entryPrice) *
100;
 double get realizedPnl => status != TradeStatus.open
 ? ((exitPrice! - entryPrice) * totalQuantity) : 0;
 bool get isProfit => unrealizedPnl > 0;
```

```
}
```

Create Hive adapter for ActiveTrade and register in main.dart.

---

## STEP 2: TRADE TRACKER PROVIDER

---

Create activeTradeProvider: StateNotifierProvider<ActiveTrade?>:

Methods:

```
void openTrade(TradeSignal signal, int lots)
 → Only if status is null (no current trade – only one trade at a time)
 → Create ActiveTrade from signal
 → Save to Hive box "active_trade" (key: "current")
```

```
void updatePrice(double newPrice)
 → Set currentPrice
 → Check SL/TP, trigger AlertRuleEngine
```

```
void closeTrade(double exitPrice, String reason)
 → Set exitPrice, exitTime, status, exitReason
 → Move to Hive box "trade_history"
 → Remove from "active_trade" box
 → Notify state null (no active trade)
```

```
void trailSL(double newSL)
 → Validate newSL > current SL (only trail upward for PE buys)
 → Update sl field
 → Save to Hive
```

```
void exitPartial(double exitPrice)
 → Reduce lots by 50% (round up)
 → Log partial exit in notes
```

Load active trade from Hive on app start (persist through app restarts).

---

## STEP 3: TRADE TRACKER SCREEN

---

lib/features/trade\_tracker/trade\_tracker\_screen.dart:

IF no active trade:

Centered card: "No Active Trade"

Subtext: "Signals from the AI will appear on the Dashboard tab"

Button: "Log Manual Trade" → opens ManualTradeBottomSheet

IF active trade exists: show ActiveTradeCard

ActiveTradeCard:

Header row: "{strike} {type}" large | "{lots} lots" badge | time elapsed "2h 14m"

P&L display (center, large):

"+₹12,450" in green / "-₹3,200" in red

"(+194.2%)" secondary

Animated: counts up/down on each price tick (use TweenAnimationBuilder)

Price row:

Entry: ₹62 → CMP: ₹181 (arrow between them)

SL/TP progress bar:

Linear progress bar showing current price position between SL and TP2

Color: red zone (SL to entry), yellow zone (entry to TP1), green zone (TP1 to TP2)

Markers: SL pin, TP1 pin, TP2 pin, current price pin

Action buttons row:

[Exit 50%] [Exit 100%] [Trail SL] [Edit SL/TP]

"Exit 50%" → confirm dialog → closeTrade with half quantity

"Exit 100%" → confirm dialog with reason dropdown → closeTrade

"Trail SL" → shows input field pre-filled with current SL → trailSL()

"Edit SL/TP" → shows bottom sheet with SL, TP1, TP2 text fields

ManualTradeBottomSheet:

- Strike (number input)
- Option type toggle (PE / CE)
- Entry price, Lots, SL, TP1, TP2
- "Log Trade" button
- Validates SL < Entry for both PE and CE

---

## STEP 4: P&L DASHBOARD SCREEN

---

lib/features/pnl/pnl\_screen.dart:

Section 1 – Summary Cards (2x2 grid):

"This Week P&L": total realized P&L in green/red

"Win Rate": X% (wins / total trades)

"Best Trade": +₹X (highest single trade profit)

"Worst Trade": -₹X (worst single trade loss)

## Section 2 – Daily P&L Bar Chart (fl\_chart BarChart):

Show last 10 trading days

Green bars: profitable day, Red bars: losing day

X-axis: "Mon", "Tue" etc.

Y-axis: ₹ amount

Tap bar: show tooltip with that day's total P&L

## Section 3 – Trade History Table:

Columns: Date | Strike | Type | Entry | Exit | P&L | P&L%

Sorted newest first

Color code P&L column: green if positive, red if negative

Tap row: expand to show full trade details + which AI signal triggered it

## Section 4 – Risk Stats:

Average profit: ₹X

Average loss: ₹X

Risk-reward ratio: 1 : X.X

Largest drawdown: -X%

## EXPECTED OUTPUT:

- Full trade lifecycle: log → update live → exit → appears in P&L history
- P&L updates every tick when trade is active (animated number)
- SL/TP progress bar shows current position correctly
- P&L dashboard computes all stats from Hive trade history
- Partial exit logs correctly

# PHASE 8 — Settings, Backtesting & Debug Tools

TASK: Build the Settings screen with full app configuration, a signal accuracy backtest panel, and a developer debug log viewer.

Phases 1-7 complete. All core features working end-to-end.

---

## STEP 1: APP SETTINGS MODEL & PERSISTENCE

---

Create lib/data/models/app\_settings.dart:

```
class AppSettings {
 // API keys (override --dart-define values at runtime for convenience)
 String dhanAccessToken;
 String geminiApiKey;
```

```

// Signal engine
int analysisIntervalMinutes; // 1, 5, or 15
TimeOfDay tradingStartTime; // default 09:30
TimeOfDay tradingEndTime; // default 14:30
String geminiModel; // "gemini-2.5-flash" or "gemini-2.5-pro"
int maxSignalsPerDay; // 1-30, default 20

// Risk management
int defaultLots;
double maxLossPercent; // e.g. 15.0 means exit if trade loses 15%

// Notification preferences
bool notifyOnBuySignal; // default true
bool notifyOnExitSignal; // default true
bool notifyOnTrailSL; // default true
bool notifyOnSLBreach; // default true
bool notifyOnTPHit; // default true
bool playSounds; // default true

// Theme
bool useDarkTheme; // default true
}

```

Persist to SharedPreferences as JSON.

settingsProvider: StateNotifierProvider<AppSettings> – auto-saves on any change.

---

## STEP 2: SETTINGS SCREEN

---

lib/features/settings/settings\_screen.dart – use ListView with sections:

### Section "API Configuration":

- Dhan Access Token: masked text field + eye toggle + "Test Connection" button  
→ On tap: DhanApiClient.getCurrentPrice() → show  latency or  error
- Gemini API Key: masked field + "Test" button  
→ On tap: send minimal test prompt → show  or 
- Estimated Gemini cost: "₹{X} today | ₹{Y} this month"  
(calculate from call count × 0.075 per 1M tokens estimate)

### Section "Signal Engine":

- Analysis interval: SegmentedButton [1m / 5m / 15m]
- Trading hours: two TimePickerDialog launchers (Start / End)
- Gemini model: DropdownButton [Flash (faster, cheaper) / Pro (smarter)]
- Max signals/day: Slider 5→30

#### Section "Risk Parameters":

- Default lots: number stepper (+/-)
- Auto-alert at loss %: Slider 5%→30%
- [Future] Auto-exit toggle (disabled, labeled "Coming Soon")

#### Section "Notifications":

- Individual SwitchListTile for each notification type
- Master mute: if off, all sub-toggles gray out

#### Section "Display":

- Theme toggle Dark/Light

#### Section "About & Legal":

- App version
- SEBI Disclaimer (non-dismissible text):  
"This app is for personal research only. It does not constitute financial advice. The developer is not a SEBI-registered analyst. Trade at your own risk."
- "Clear All Data" button → confirmation dialog → clears Hive +

SharedPreferences

---

### STEP 3: SIGNAL BACKTEST PANEL

---

lib/features/signals/backtest\_panel.dart (accessible from Signals screen via tab):

For each past BUY signal in Hive (last 30 days):

Show what price moved to in the 30 minutes after signal:

Signal: "BUY 23800 PE @ ₹185 | SL: ₹163 | TP1: ₹215"

Outcome: "TP1 Hit (+16.2%)" in green OR "SL Hit (-11.9%)" in red OR "Unknown (no trade)"

Accuracy summary card:

"AI Accuracy: X% (last 30 signals)"

"Would-be profit if all taken: +₹X"

"Average signal delay to TP1: X min"

Note: backtest uses actual price data from Hive only – no simulated data.  
If price data for a signal period isn't available, show "Insufficient data".

---

### STEP 4: DEBUG LOG VIEWER

---

lib/core/utils/app\_logger.dart:

Levels: verbose, debug, info, warn, error



Write to Hive box "app\_logs" (max 500 entries, FIFO)  
Each entry: {level, message, timestamp, context}

Settings > "View Debug Logs" → opens LogViewerScreen:

Filter by level (chips)  
Reverse chronological order  
Error logs in red, warn in amber, info in white  
"Copy All Logs" button (copies to clipboard)  
"Clear Logs" button

EXPECTED OUTPUT:

- All settings persist after app restart
- Test Connection buttons show real latency in <3 seconds
- Settings changes (interval, model) immediately affect SignalScheduler behavior
- Debug log viewer shows all app events since install

---

## PHASE 9 — Error Handling, Resilience & Background Service

TASK: Harden the entire app for real-world use. The app must survive network loss, API failures, Gemini errors, app backgrounding, and device restart gracefully. Zero unhandled exceptions during a 6-hour market session.

All previous 8 phases complete and functional.

---

### STEP 1: EXCEPTION HIERARCHY

---

Create lib/core/utils/app\_exceptions.dart:

```
abstract class AppException implements Exception {
 final String message;
 final String? context;
 final DateTime timestamp;
}

class NetworkException extends AppException { }
class AuthException extends AppException { } // 401 from Dhan
class RateLimitException extends AppException { } // 429 from any API
class ServerException extends AppException { } // 5xx
class ParseException extends AppException { // Gemini JSON parse fail
 final String rawResponse;
}
class WebSocketException extends AppException { }
```

```
class TradeValidationException extends AppException { }
```

All exceptions: override toString() with user-friendly message (no stack traces)

---

## STEP 2: GLOBAL ERROR HANDLING

---

In main.dart:

```
FlutterError.onError = (FlutterErrorDetails details) {
 AppLogger.error('Flutter error', details.exceptionAsString(),
 details.context.toString());
 // DO NOT crash – log and continue
};

PlatformDispatcher.instance.onError = (error, stack) {
 AppLogger.error('Platform error', error.toString());
 return true; // handled
};

ErrorWidget.builder = (FlutterErrorDetails details) {
 return Scaffold(
 body: Center(
 child: Column(children: [
 Icon(Icons.warning, color: Colors.amber),
 Text("Something went wrong. The app will continue."),
 TextButton(onPressed: () => /* reset route */, child: Text("Go to
Dashboard")),
]),
),
);
};
```

---

## STEP 3: CONNECTIVITY RESILIENCE

---

Create lib/core/services/connectivity\_service.dart using connectivity\_plus:

```
Stream<bool> isConnectedStream (true/false)
```

```
isConnectedProvider: StreamProvider<bool>
```

In the app shell: watch isConnectedProvider, show a persistent amber banner "No internet connection – prices paused" when offline. Auto-dismiss when reconnected.

On reconnect (transition false → true):

1. Re-initialize WebSocket connection
2. Fetch latest candles to fill any gap
3. Resume SignalScheduler
4. Show brief green snackbar: "Reconnected – live data restored"

---

#### STEP 4: ALL PROVIDERS – ERROR STATES WITH RETRY UI

---

Every provider that makes a network call must:

1. Catch all `AppException` subclasses
2. Emit `AsyncError(exception)` state
3. The consuming widget must handle `.when(data:, loading:, error:)`

Create a reusable `ErrorRetryWidget`:

Icon (warning), message text, "Retry" button

On retry: `ref.invalidate(provider)`

Use this widget in: `DashboardScreen`, `SignalsScreen`, `PnLScreen` for their respective data providers.

---

#### STEP 5: DATA VALIDATION LAYER

---

In `DhanApiClient.getHistoricalCandles()`:

After parsing, run `CandleValidator`:

- Reject candles where volume = 0 and market was open
- Reject candles where price differs >5% from adjacent candle (likely bad data)
- Reject candles with timestamp in the future or before today's session
- Log rejected candle count to `AppLogger`

In `SignalParser.parse()`:

After parsing Gemini JSON:

- Reject if `SL` >= entry price for any BUY signal
- Reject if strike is outside NIFTY ±500 from current price
- Reject if confidence is not HIGH/MEDIUM/LOW
- On rejection: log full Gemini response, throw `ParseException`

---

#### STEP 6: BACKGROUND EXECUTION

---

Use `flutter_background_service` or `workmanager` to keep analysis running:

Android:

- Foreground service with persistent notification:  
"NIFTY Trader – Monitoring market (next signal in Xm)"  
Required for reliable background execution on Android 8+
- WorkManager for 15-min fallback if service is killed

iOS:

- Background fetch with BGTaskScheduler
- Limited to 30s execution windows
- Show notification: "Open app to see latest signal" when backgrounded

State persistence on app kill + restart:

- Active trade: already in Hive (Phase 7)
- Signal history: already in Hive (Phase 4)
- Last analysis time: save to SharedPreferences
- On restart: if market is open, immediately start scheduler

EXPECTED OUTPUT:

- Network loss → offline banner appears, prices pause
- Network restore → reconnects automatically, no user action needed
- API error → retry button appears, never crashes
- Gemini bad JSON → ParseException logged, scheduler continues to next cycle
- App kill + restart → active trade and signal history restored from Hive
- Background notification shows while app is minimized during market hours

---

## PHASE 10 — Polish, Onboarding, Testing & Release Build

TASK: Final production polish. Onboarding flow, UI animations, performance profiling, unit + widget tests, and a signed release APK. After this phase, the app is complete, stable, and ready for daily use during market hours.

All phases 1-9 complete. App is functionally complete.

---

STEP 1: ONBOARDING FLOW (shown only on first launch)

---

Implement `lib/features/onboarding/onboarding_screen.dart` with `PageView`:

Page 1 – Welcome:

- App logo, tagline, feature highlights (3 bullet points)
- Large animated SEBI disclaimer (user must scroll to bottom to enable "Next")

## Page 2 – Dhan API Setup:

"Enter your Dhan Access Token"

Text field (masked, paste-friendly)

"How to get this" expandable (step-by-step with dhan.co URL)

"Test Connection" button → must show  before "Next" is enabled

## Page 3 – Gemini API Setup:

"Enter your Gemini API Key"

Text field

"Get it free at aistudio.google.com" link

"Test Connection" → must show  before "Next" is enabled

## Page 4 – Risk Settings:

"Set your defaults"

Default lots stepper

Max loss % slider

Trading hours pickers

## Page 5 – Ready:

"You're all set! 

Market open status

"Go to Dashboard" button

On complete: set SharedPreferences key "onboarding\_complete" = true

Check this key in main.dart router – if false, redirect to /onboarding

---

## STEP 2: UI POLISH & ANIMATIONS

---

Add these specific animations – no others:

1. Price flash animation (already in Phase 6, verify it's smooth)
  - Use AnimatedContainer with color transition, 400ms
2. P&L countup animation:
  - Use TweenAnimationBuilder<double> on unrealizedPnl
  - Duration: 300ms
  - Only animate if difference > ₹10 (don't animate micro-ticks)
3. Signal card entrance animation:
  - When new signal added to list: SlideTransition from right + FadeTransition
  - Duration: 250ms
4. Shimmer loading state:
  - Use shimmer package for all loading states
  - Match shimmer shape to actual widget layout

- Dashboard: shimmer chart + shimmer signal card

#### 5. Haptic feedback:

- `HapticFeedback.heavyImpact()` on new BUY signal
- `HapticFeedback.mediumImpact()` on new EXIT signal
- `HapticFeedback.lightImpact()` on button taps

#### 6. Bottom nav badge:

- Show red dot on "Signals" tab when there's an unread signal

---

### STEP 3: PERFORMANCE OPTIMIZATION

---

#### 1. WebSocket price debouncing:

Add to `livePriceProvider`: only propagate UI update if price changed by  $\geq ₹0.50$   
OR if more than 250ms has elapsed since last update.  
(Prevents rebuilding chart on every sub-paisa tick)

#### 2. Chart optimization:

Only rebuild `CandlestickChart` when candle list length changes  
(new candle added) – not on every price tick.

Use `select()` in Riverpod: `ref.watch(historicalCandlesProvider.select((c) => c.length))`

#### 3. Indicator computation:

Already in `compute()` isolate (Phase 3). Verify it doesn't block main thread.  
Profile with Flutter DevTools – target <8ms for indicator calculations.

#### 4. Hive optimization:

Use `compactHiveBoxes()` on app start to clean up deleted records.  
Limit `trade_history` box to 365 entries (delete oldest on overflow).

#### 5. Const constructors:

Run: `flutter analyze` and fix all "prefer\_const\_constructors" warnings.  
Target: 0 analyzer warnings.

---

### STEP 4: TESTS

---

#### Unit tests (test/):

`indicator_service_test.dart` – already written in Phase 3, run and verify pass

`signal_parser_test.dart`:

- parse valid JSON for all 7 action types → no exception
- parse JSON missing "action" → throws `ParseException`

- parse JSON with SL > entry → throws ParseException
  - parse markdown-wrapped JSON (``json...``) → strips and parses correctly
- alert\_rule\_engine\_test.dart:
- SL breach fires exactly once per trade
  - TP1 fires exactly once per trade
  - Alerts reset when trade is closed

Widget tests (test/widget\_test.dart):

- Dashboard renders with mock providers (no real API calls)
- ActiveTradeCard shows correct P&L given entry=100, current=150
- SignalCard renders all 7 action types without overflow
- LatestSignalCard shows "No signals yet" when list is empty

Run all: flutter test – target 0 failures.

---

## STEP 5: RELEASE BUILD

---

android/app/build.gradle:

```
applicationId: "com.personal.niftyaitrader"
versionCode: 1
versionName: "1.0.0"
minSdkVersion: 23 (Android 6+)
targetSdkVersion: 34
```

Enable R8 (minification):

```
buildTypes { release { minifyEnabled true; shrinkResources true } }
```

ProGuard rules for packages:

```
-keep class io.flutter.** { *; }
-keep class com.google.firebase.** { *; }
-keep class com.dhan.** { *; }
```

Create keystore and signing config (provide instructions, not the actual keys):

```
keytool -genkey -v -keystore nifty_trader.jks -alias nifty -keyalg RSA -keysize
2048 -validity 10000
```

Build command:

```
flutter build apk --release \
 --dart-define=DHAN_CLIENT_ID=YOUR_ID \
 --dart-define=DHAN_ACCESS_TOKEN=YOUR_TOKEN \
 --dart-define=GEMINI_API_KEY=YOUR_KEY
```

---

## STEP 6: FINAL SAFETY CHECKLIST

---

Verify ALL of the following are true:

- ❑ `grep -r "AIzaSy" lib/` → returns nothing (no hardcoded Gemini key)
- ❑ `grep -r "access_token" lib/` → returns nothing (no hardcoded Dhan token)
- ❑ `flutter analyze` → 0 errors, 0 warnings
- ❑ `flutter test` → all tests pass
- ❑ App runs for 30 minutes with real keys without crash
- ❑ Network off → offline banner, no crash
- ❑ Network back on → reconnects automatically
- ❑ New signal arrives → notification shows on lock screen
- ❑ Active trade P&L animates smoothly
- ❑ SEBI disclaimer visible on onboarding page 1 (not skippable)
- ❑ SEBI disclaimer text also visible in Settings > About
- ❑ Release APK installs on Android 8+ device
- ❑ APK size < 35MB (run `flutter build apk --analyze-size`)

EXPECTED FINAL OUTPUT:

- A fully functional, production-quality Flutter app that:
- Connects to Dhan WebSocket for real-time NIFTY price
  - Runs Gemini AI analysis every 5 minutes during market hours
  - Sends push notifications for every actionable signal
  - Tracks active trades with live P&L
  - Persists all data through app restarts
  - Runs stably for a full 6-hour market session
  - Zero unhandled exceptions in normal use

QUICK REFERENCE — Jules Usage Tips

| Situation                                    | What to do                                                                       |
|----------------------------------------------|----------------------------------------------------------------------------------|
| Jules gets stuck mid-task                    | Type "Focus on Step X only and skip the rest"                                    |
| Jules breaks a previous phase                | Start a new task: "Revert changes to file X and redo only Y"                     |
| Jules adds extra features you didn't ask for | "Remove everything you added except the specific files listed in Step N"         |
| Test fails                                   | New task: "Fix this specific test failure: [paste error]"                        |
| PR is too large                              | Ask Jules to split into multiple smaller PRs by step                             |
| Gemini JSON parse fails in production        | New task: "Add raw response logging to SignalParser and improve JSON extraction" |



# Gemini Model Reference

| Model            | Use in app for                     | Speed  | Cost        |
|------------------|------------------------------------|--------|-------------|
| gemini-2.5-flash | Signal analysis (every 5 min)      | Fast   | ~₹0.04/call |
| gemini-2.5-pro   | Use in Settings as "Accurate mode" | Slower | ~₹0.40/call |

At 5-min intervals:  $\sim 72 \text{ calls/market day} \times ₹0.04 = \sim \textbf{₹3/day}$  with Flash.